

CUNEF UNIVERSIDAD

SmartGrid-ES

A Custom Gymnasium Environment for Reinforcement Learning in a Simplified Smart Grid

Final Academic Report – Version V3

Authors: Jaime Blanco-González
Alonso Castro-Freibott
Javier-Ignacio Fernández-Madrigal
Subject: Artificial Intelligence
Project: SmartGrid-ES
Main version: V3
Optional extension: V3.1 PSO reward-weight tuning
Date: May 18, 2026

Academic reinforcement learning project based on a custom Gymnasium environment, tabular Q-learning, synthetic weekly scenarios, a `scikit-fuzzy` risk module and scenario-based evaluation.

Contents

1	Abstract	1
2	Introduction	1
3	Version evolution: V1, V2 and V3	1
4	Methodology	2
5	V3 environment design	3
5.1	Environment overview	3
5.2	Observation space	3
5.3	Action space	4
5.4	Transition logic and episode termination	4
5.5	Episode length: 168 hourly steps	5
5.6	Training mode and evaluation mode	5
6	Data and scenarios	5
7	Q-learning agent	6
8	Reward design	7
9	Fuzzy energy-risk module	8
10	Training and evaluation setup	9
11	Results	9
11.1	Evaluation metrics	9
11.2	Performance metrics	10
11.3	Energy and risk metrics	11
11.4	Interpretation of the V3 results	12
12	Demo episode analysis	12
13	Dashboard	13
14	V3.1 PSO reward-weight tuning	13
14.1	PSO configuration and bounds	14
14.2	Best reward weights and metrics	15
15	Course concepts covered	16
16	Limitations	16

17 Conclusions	17
18 Future work	17
A Execution commands	17
B Pinned dependencies	18
C Repository structure relevant to V3	19
D Final artifacts and reproducibility note	19
E Main files and artifacts used	20
References	20

1 Abstract

SmartGrid-ES is an academic Artificial Intelligence project that applies reinforcement learning to a simplified microgrid management problem. The project evolved through three milestones. V1 demonstrated the basic reinforcement-learning cycle using a predefined Gymnasium environment. V2 introduced a wrapper-based smart-grid abstraction. V3, the final main version, replaces the wrapper-centered approach with a dedicated custom Gymnasium environment, **SmartGridEnvV3**, driven by JSON scenario definitions and synthetic CSV time series.

The V3 environment models a discrete and controlled microgrid abstraction with battery storage, demand, renewable generation, price, time-of-day period and simplified weather conditions. A tabular Q-learning agent is trained over weekly synthetic episodes of 168 hourly steps. Evaluation is performed by scenario using greedy policies, aggregate metrics, demo episodes and a Streamlit dashboard. The project also includes a fuzzy energy-risk module used as an evaluation metric, reward penalty and dashboard signal. An optional V3.1 experiment connects Particle Swarm Optimization (PSO) to the reward-design problem by searching for candidate reward-weight vectors. PSO does not replace Q-learning and does not directly control the environment.

The reported results are taken from the final artifacts available in **results/v3/** and **results/v3_1/**. They show meaningful differences across the five V3 scenarios. The **renewable_volatility** scenario obtains the highest average reward in the final 2026-05-18 evaluation summary, closely followed by **summer_surplus**, while **winter_peak** is the most costly due to high grid purchases and high fuzzy risk. These results should be interpreted within the project scope: the data are synthetic, discrete and controlled, and the environment is an academic abstraction rather than a real electricity-market simulator.

2 Introduction

Reinforcement Learning (RL) studies how an agent can learn by interacting with an environment, selecting actions, receiving rewards and gradually improving its policy [1]. A simplified smart-grid-inspired microgrid is a suitable academic setting for this idea because the agent must repeatedly decide how to use storage, renewable generation and external grid purchases under changing demand and price conditions.

The objective of SmartGrid-ES is not to reproduce the physical or market behavior of the Spanish electricity system. Instead, it provides a controlled educational environment where the concepts of agent, state, action, reward, policy, Q-table, scenario evaluation and uncertainty-aware decision support can be implemented and inspected. This makes the project appropriate for a university AI course: the model is small enough to remain interpretable, but rich enough to compare behavior across different operating scenarios.

V3 contributes a more coherent technical architecture than the previous milestones. Rather than modifying a base environment through wrappers, the final version introduces a custom Gymnasium environment whose state variables, action space, transition logic, reward function, input data and evaluation outputs are explicit. This improves reproducibility, allows scenario definitions to be stored outside the Python code and makes the results easier to inspect through CSV summaries, demo episodes and a dashboard.

3 Version evolution: V1, V2 and V3

The repository preserves V1 and V2 as historical milestones, but V3 is the principal final delivery. This distinction matters because the project was developed progressively: first proving that the team could implement an RL loop, then introducing smart-grid-like wrappers, and finally building a

custom environment aligned with the project domain.

Table 1: Evolution of the SmartGrid-ES project.

Version	Main idea	Contribution	Limitations
V1	Predesigned Gymnasium environment	Basic reinforcement-learning cycle	Not specific to the SmartGrid-ES domain
V2	Wrapper-based smart-grid abstraction	Scenario variation through environment wrappers	Still dependent on wrapper modifications around a base environment
V3	Custom Gymnasium environment	Dedicated smart-grid environment with JSON/CSV scenarios, fuzzy risk and extended evaluation	Simplified synthetic and discrete setting

In V2, the base environment represented a smaller discrete state (**battery**, **demand**, **renewable**, **period**) and four actions. Scenario variation was introduced through wrappers such as season, demand noise, battery loss and reward shaping. V3 moves the main project logic into **SmartGridEnvV3**, expands the observation with price and weather, separates scenario configuration into JSON files and uses synthetic time-series CSV files for both training and evaluation.

4 Methodology

The V3 methodology follows a reproducible pipeline from scenario definition to final visualization:

1. Define scenario parameters in JSON files under **data/scenarios/**.
2. Load synthetic training and evaluation time series from CSV files under **data/timeseries/**.
3. Construct **SmartGridEnvV3** in either training mode or evaluation mode.
4. Train a tabular Q-learning agent using epsilon-greedy exploration.
5. Save the learned Q-table under **results/v3/models/**.
6. Evaluate the trained agent greedily on the scenario evaluation CSV.
7. Save aggregate summaries, plots and demo episodes under **results/v3/**.
8. Inspect the results through the Streamlit dashboard.

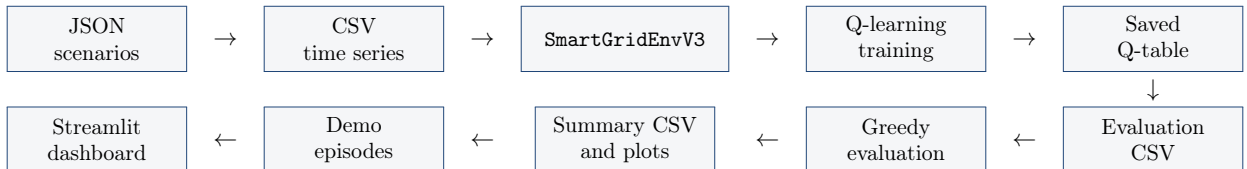


Figure 1: V3 experimental pipeline from external scenario definition to final dashboard inspection. Configuration, learning, evaluation and output artifacts remain separated to support reproducibility.

This methodology emphasizes interpretability and reproducibility. The scenarios are explicit, the state and action spaces are discrete, the agent is tabular and the output artifacts are stored in inspectable CSV and image files.

5 V3 environment design

5.1 Environment overview

`SmartGridEnvV3` is a custom Gymnasium environment [3] representing a simplified microgrid. It receives a JSON scenario path, selects the training or evaluation CSV depending on the selected mode, loads the time series and exposes a standard Gymnasium interaction loop through `reset()` and `step(action)`.

From a Markov Decision Process perspective, the V3 environment defines a finite state space, a finite action space, transition dynamics driven by the time-series row and battery state, and a scalar reward function. The Markov framing is simplified: the current observation contains the discrete variables used by the agent, while the next external conditions are read from the controlled scenario sequence.

The environment is intentionally discrete. This choice keeps the state space small enough for tabular Q-learning and allows the learned Q-table to be inspected directly. The environment does not include fuzzy risk as part of the observation. Instead, risk is computed from the current state and exposed through the `info` dictionary, where it can be used for evaluation, reward calculation and dashboard visualization.

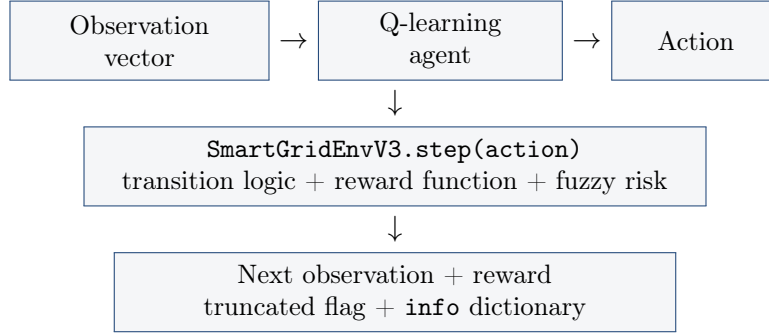


Figure 2: Agent-environment interaction in V3. The observation vector is `[battery, demand, renewable, price, period, weather]`. Fuzzy risk is not part of that observation vector; it is computed inside the environment step, exposed through `info` and used by the reward, evaluation metrics and dashboard.

5.2 Observation space

The V3 observation is:

$$[battery, demand, renewable, price, period, weather]$$

The Gymnasium observation space is a `MultiDiscrete` space with shape:

$$[5, 4, 4, 3, 4, 3]$$

This corresponds to battery levels 0..4, demand levels 0..3, renewable levels 0..3, price levels 0..2, day periods 0..3 and weather levels 0..2.

Table 2: V3 observation variables.

Variable	Range	Meaning	Why it matters
battery	0..4	Stored energy level	Indicates storage availability for demand coverage.
demand	0..3	Electricity demand level	Represents consumption pressure in the current hour.
renewable	0..3	Renewable generation level	Represents available local clean energy.
price	0..2	Electricity price level	Adds cost pressure when external grid energy is used.
period	0..3	Time-of-day period	Captures daily cycle structure using four six-hour blocks.
weather	0..2	Simplified weather condition	Provides contextual information related to renewable availability.

5.3 Action space

The action space is `Discrete(5)`. Each action represents a simplified energy-management decision.

Table 3: V3 action space.

Action ID	Action	Meaning
0	Use battery	Use stored energy to cover demand that is not satisfied by renewable generation.
1	Buy from grid	Buy missing energy from the external grid when local supply is insufficient.
2	Store surplus	Store renewable surplus in the battery when capacity is available.
3	Sell surplus	Sell unused renewable energy when there is surplus.
4	Hold	Take no active management action.

5.4 Transition logic and episode termination

At each step, the environment reads the current row of the synthetic time series. Renewable generation first covers demand directly. The selected action then modifies the energy balance by using the battery, buying from the grid, storing surplus, selling surplus or doing nothing. Invalid or useless actions are marked in the `info` dictionary and penalized by the reward function.

The environment never terminates through a success or failure terminal state in the V3 implementation. Instead, each episode is truncated when the end of the scenario window is reached. The JSON scenario configuration sets `max_steps` to 168.

5.5 Episode length: 168 hourly steps

Each V3 episode contains:

$$7 \text{ days} \times 24 \text{ hours per day} = 168 \text{ steps}$$

An episode therefore represents one synthetic week of microgrid operation. This design captures repeated daily cycles in demand, renewable generation, price and simplified weather. It also keeps evaluation interpretable: one greedy evaluation episode corresponds to one complete weekly sequence. The design does not attempt to model long-term grid operation, seasonal planning or real-world market dispatch.

5.6 Training mode and evaluation mode

The environment supports two modes: **training mode**, which uses the scenario training CSV, and **evaluation mode**, which uses the scenario evaluation CSV. This separation makes it possible to train on one synthetic weekly sequence and evaluate on a different controlled sequence for the same scenario.

6 Data and scenarios

The V3 data are synthetic, discrete and controlled. They are not real-world measurements and they are not live data. Each scenario has a JSON file that defines the scenario name, mode, training CSV, evaluation CSV, battery parameters, maximum episode length and reward weights. The scenario loader validates required fields and accepts the implemented `csv` mode.

Each CSV contains six discrete columns. The time-series loader validates that these columns exist, are integer-valued and remain within their allowed ranges.

Table 4: Columns used by the V3 synthetic time-series CSV files.

Column	Range	Meaning
step	≥ 0	Hourly index within the synthetic weekly sequence.
hour	0..23	Hour of the day, used to derive the four time-of-day periods.
weather_level	0..2	Simplified weather condition.
demand_level	0..3	Discrete electricity demand level.
renewable_level	0..3	Discrete renewable generation level.
price_level	0..2	Discrete electricity price level.

Table 5: Common configuration shared by the V3 scenarios.

Configuration item	Value in V3 scenarios
Scenario mode	csv
Training input	One synthetic CSV time series per scenario
Evaluation input	One synthetic CSV time series per scenario
Battery capacity	4 discrete levels above zero, represented as battery range 0..4
Initial battery	2
Episode length	168 hourly steps
Reward-weight structure	Shared across scenarios: covered demand, unmet demand, grid bought, sold energy, invalid action, wasted renewable and fuzzy risk

All V3 scenarios use the same reward-weight structure and the same numerical weights in the final scenario JSON files. This is important for comparison: the scenarios are evaluated under a common objective, so the differences observed in the results are mainly attributable to the demand, renewable, price and weather patterns of each scenario, rather than to changing the reward function between scenarios.

Table 6: V3 scenarios.

Scenario	Purpose
baseline_v3	Reference balanced scenario.
winter_peak	High demand and price pressure.
summer_surplus	Higher renewable surplus.
grid_stress	Demand peaks and limited renewable support.
renewable_volatility	Unstable renewable generation and price variation.

Each scenario uses a weekly window of 168 hourly steps for training and another weekly window of 168 hourly steps for evaluation.

7 Q-learning agent

The learning agent is a tabular Q-learning agent. For each discrete state-action pair, it stores a Q-value estimating the expected discounted return of taking a given action in a given state and then following the learned policy [2]. Because the V3 state space is small and fully discrete, tabular Q-learning is appropriate for the educational scope of the project.

The update rule is:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

where α is the learning rate, γ is the discount factor, r is the received reward, s is the current state, a is the chosen action and s' is the next state.

During training, the agent uses an epsilon-greedy strategy. With probability ϵ , it explores by choosing a random action. With probability $1 - \epsilon$, it exploits the current Q-table by choosing the action with the highest Q-value. During evaluation, $\epsilon = 0$, so the policy is greedy.

Deep reinforcement learning is not used. This is a deliberate design choice: the state representation is discrete and compact, and the main academic value of the project is interpretability rather than scalability to high-dimensional observations.

Table 7: V3 Q-table shape.

Dimension	Size	Meaning
Battery	5	Battery levels 0..4
Demand	4	Demand levels 0..3
Renewable	4	Renewable levels 0..3
Price	3	Price levels 0..2
Period	4	Four six-hour periods
Weather	3	Weather levels 0..2
Action	5	Five discrete actions

The resulting Q-table has shape (5, 4, 4, 3, 4, 3, 5). The first six dimensions correspond to the discrete observation and the final dimension stores the five action values available in each state.

Table 8: V3 Q-learning hyperparameters from `src/training/config.py`.

Parameter	Value	Meaning
<code>seed</code>	42	Reproducibility seed.
<code>episodes</code>	3000	Base number of training episodes.
<code>alpha</code>	0.15	Q-learning update rate.
<code>gamma</code>	0.95	Discount factor for future rewards.
<code>epsilon</code>	1.0	Initial exploration probability.
<code>epsilon_min</code>	0.05	Minimum exploration probability.
<code>epsilon_decay</code>	0.997	Multiplicative decay applied after each episode.
<code>log_every</code>	100	Training logging interval.

The harder V3 scenarios `winter_peak` and `grid_stress` override the base training configuration with 4000 episodes and `epsilon_decay = 0.998`. V3 does not use `max_steps_per_episode` from the older V2 configuration; the episode length is read from the scenario JSON.

8 Reward design

The V3 reward is a weighted and interpretable function of energy-management outcomes. It is not a real electricity-market settlement function. Its goal is to guide the agent toward reasonable behavior in the simplified environment: cover demand, avoid unmet demand, reduce costly grid purchases, use renewable surplus productively and avoid high-risk states.

Table 9: Reward components in V3.

Component	Effect	Purpose
<code>demand_covered</code>	Positive	Reward satisfying demand.
<code>unmet_demand</code>	Negative	Penalize failure to serve demand.
<code>grid_bought</code>	Negative	Penalize external energy cost, with price-sensitive grid cost.
<code>sold</code>	Positive	Reward selling surplus.
<code>invalid_action</code>	Negative	Discourage useless or impossible actions.
<code>wasted_renewable</code>	Negative	Discourage wasting renewable surplus.
<code>risk</code>	Negative	Discourage high-risk states.

The reward implementation can be written as the following step-level expression:

$$R_t = w_c \cdot \text{demand_covered}_t + w_u \cdot \text{unmet_demand}_t + w_g \cdot \text{grid_cost}_t \\ + w_s \cdot \text{sold}_t + w_i \cdot \text{invalid_action}_t + w_w \cdot \text{wasted_renewable}_t + w_r \cdot \text{risk_score}_t.$$

The grid-purchase term is price-sensitive:

$$\text{grid_cost}_t = \text{grid_bought}_t \cdot (1 + \text{price_level}_t).$$

The final V3 scenario JSON files use the following shared reward weights: `demand_covered` = 2.0, `unmet_demand` = -5.0, `grid_bought` = -0.8, `sold` = 0.8, `invalid_action` = -1.5, `wasted_renewable` = -0.5 and `risk` = -0.03. Using a common reward criterion makes the scenario comparison more defensible because the learned policies are assessed under the same objective structure.

9 Fuzzy energy-risk module

The fuzzy energy-risk module estimates a risk signal from four input variables: battery level, demand level, renewable generation level and price level. It outputs `risk_score`, a continuous score from 0 to 100; `risk_level`, a discrete category 0, 1 or 2; and `risk_level_name`, which is `low`, `medium` or `high`.

In the final V3 branch, this module is implemented with `scikit-fuzzy`. The implementation defines `ctrl.Antecedent` variables for `battery`, `demand`, `renewable` and `price`, and a `ctrl.Consequent` variable for `risk`. Membership functions are defined with `fuzz.trimf` and `fuzz.trapmf`. The rule base is expressed through `ctrl.Rule`, assembled as a `ctrl.ControlSystem` and evaluated with `ctrl.ControlSystemSimulation`.

High risk is activated by combinations such as low battery with high demand, low renewable generation with high price, or peak demand with high price. Medium and low-risk outputs are activated by less constrained combinations, such as adequate battery level, lower demand or stronger renewable support. The implementation also keeps a deterministic fallback pressure formula in case the fuzzy simulation does not return a finite risk output.

This module connects the project with the fuzzy logic topic of the course and with the idea of fuzzy sets introduced by Zadeh [6]. Like the classical tipping example, it converts linguistic rules into a numeric output. In SmartGrid-ES, the output is not a control action by itself. It is used in three

ways: as an evaluation metric, as a penalty in the reward function and as a visual signal in the dashboard.

10 Training and evaluation setup

Each V3 episode represents a synthetic week of operation with 168 hourly steps. During training, the same scenario training week is repeated for many episodes while the agent updates its Q-table. During evaluation, the learned agent is loaded from `results/v3/models/` and executed greedily on the scenario evaluation CSV.

The V3 evaluation configuration uses one evaluation episode. This is appropriate because each evaluation CSV is a fixed deterministic weekly sequence and a greedy policy with the same initial state produces the same trajectory.

The main generated artifacts are Q-table models in `results/v3/models/`, training plots and evaluation comparison plots in `results/v3/plots/`, demo episodes in `results/v3/demos/`, evaluation summaries in `results/v3/summaries/` and optional PSO artifacts in `results/v3_1/`.

11 Results

The V3 results in this section are taken from the final summary CSV:

`results/v3/summaries/smartgrid_v3_evaluation_summary_2026-05-18.csv`

The tables and figures in this section were prepared from the final result artifacts committed under `results/v3/` and `results/v3_1/`. No numerical result was manually invented.

11.1 Evaluation metrics

The main trajectory-level metrics are computed over the weekly evaluation trajectory. For V3, the evaluation horizon is:

$$T = 168.$$

Coverage and invalid action rate are computed as:

$$Coverage = \frac{\sum_{t=1}^T demand_covered_t}{\sum_{t=1}^T demand_t}, \quad InvalidActionRate = \frac{\sum_{t=1}^T invalid_action_t}{T}.$$

The average fuzzy risk score is:

$$AverageRisk = \frac{1}{T} \sum_{t=1}^T risk_score_t.$$

Coverage measures demand satisfaction, but it does not capture cost or risk. The risk score is a fuzzy score on a 0–100 scale, not a probability.

Table 10: Definitions of the main V3 evaluation metrics.

Metric	Definition and interpretation
Average reward	Total greedy evaluation reward averaged over the evaluation episode(s). In V3, the final evaluation uses one deterministic weekly episode per scenario.
Coverage	Share of total demand covered by renewable generation, battery use or grid purchases. It measures demand satisfaction, but not cost or risk.
Unmet demand	Total demand left uncovered during the evaluation episode. Lower is better.
Grid bought	Total energy bought from the external grid abstraction. It contributes to demand coverage but is penalized by the reward, especially at higher price levels.
Sold energy	Total surplus renewable energy sold. It can improve reward when surplus exists.
Wasted renewable energy	Renewable surplus that is neither stored nor sold. It is penalized because the simplified system prefers useful surplus management.
Battery end	Battery level at the end of the weekly evaluation episode.
Fuzzy risk score	Average risk score in the 0–100 fuzzy scale. It is not a probability and not an energy unit.
Invalid action rate	Fraction of steps in which the chosen action was not useful or possible in the current state. Lower is better.

Coverage is useful but incomplete. A scenario can have high coverage and still obtain a negative reward if the policy relies heavily on grid purchases, visits high-risk states, wastes renewable surplus or selects invalid actions. Similarly, the fuzzy risk score is shown on a 0–100 scale for readability, but it should not be interpreted as a probability or as a physical energy quantity.

11.2 Performance metrics

Table 11: V3 performance metrics by scenario. Coverage and invalid action rate are expressed as percentages.

Scenario	Avg. reward	Coverage (%)	Unmet demand	Invalid actions (%)
baseline_v3	-59.33	93.89	19.00	10.12
winter_peak	-384.81	93.68	24.00	12.50
summer_surplus	43.36	96.71	11.00	12.50
grid_stress	-68.51	95.38	15.00	10.12
renewable_volatility	48.98	97.07	9.00	13.69

11.3 Energy and risk metrics

Table 12: V3 energy and risk metrics by scenario.

Scenario	Grid bought	Sold	Wasted renewable	Battery end	Risk score
baseline_v3	129.00	20.00	3.00	0.00	55.50
winter_peak	254.00	4.00	2.00	0.00	69.43
summer_surplus	130.00	5.00	3.00	0.00	53.86
grid_stress	143.00	8.00	11.00	0.00	55.74
renewable_volatility	105.00	15.00	9.00	0.00	51.55

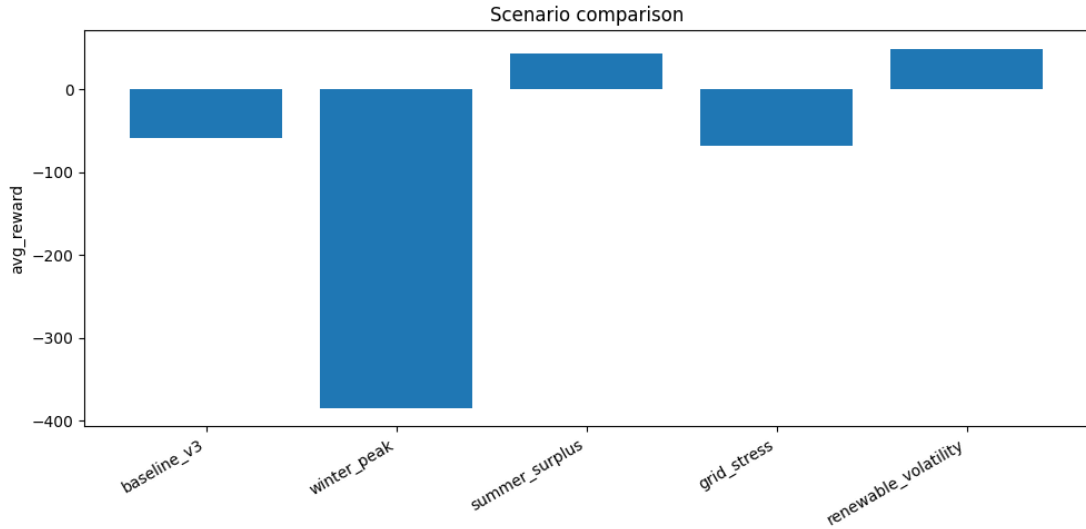


Figure 3: Average reward by V3 scenario using the final 2026-05-18 evaluation summary. The figure highlights that **winter_peak** is the most costly scenario, while **renewable_volatility** and **summer_surplus** are the positive-reward scenarios.

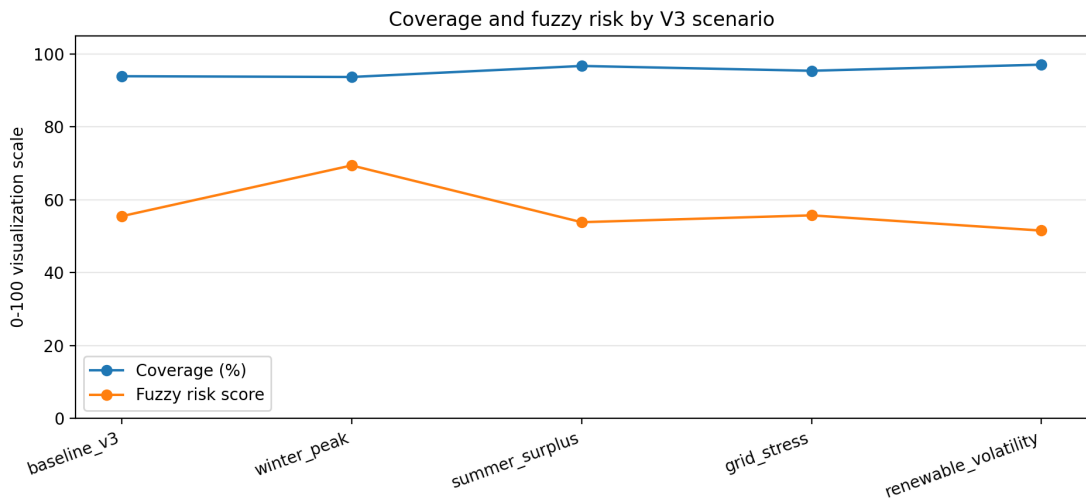


Figure 4: Coverage and fuzzy risk across V3 scenarios. Coverage (%) and fuzzy risk score are both shown on a 0–100 scale for visualization, but they are different metrics and should not be interpreted as the same physical unit.

11.4 Interpretation of the V3 results

The best average reward in the final 2026-05-18 evaluation summary is obtained by **renewable_volatility**, with an average reward of 48.98. This scenario also has the highest coverage, 97.07%, and the lowest average fuzzy risk score, 51.55. Although it has the highest invalid action rate, 13.69%, its lower grid purchases and higher sold surplus help the final reward remain positive.

summer_surplus is the second positive-reward scenario, with an average reward of 43.36 and coverage of 96.71%. It benefits from renewable availability and surplus-management opportunities, but its final reward is slightly below **renewable_volatility** because it buys more grid energy and sells less surplus in the final evaluation summary.

The most difficult scenario is **winter_peak**. It has the lowest reward, -384.81, the highest grid purchases, 254.00 units, and the highest fuzzy risk score, 69.43. Even though its demand coverage remains high at 93.68%, the reward is strongly negative because the reward function penalizes costly grid purchases, risk and invalid actions. This illustrates why coverage alone is not enough to evaluate the policy.

baseline_v3 and **grid_stress** have similar negative rewards, -59.33 and -68.51 respectively. **grid_stress** achieves slightly higher coverage, but it also has more grid purchases and more wasted renewable energy. Both scenarios show that the agent can maintain high demand coverage while still accumulating reward penalties.

Overall, the learned greedy policies maintain high demand coverage across all five scenarios. However, reward quality and risk exposure depend strongly on scenario conditions. The results show that demand coverage alone is not sufficient to evaluate the learned policy. Scenarios with high coverage can still obtain negative rewards when grid purchases, fuzzy risk, wasted renewable energy or invalid actions accumulate.

12 Demo episode analysis

The demo episodes are greedy trajectories saved after training. They are useful because they show not only aggregate metrics but also step-by-step decisions. The full demo CSV for a scenario contains 168 rows, one per hourly step. This report does not reproduce all rows; instead, Table 13 shows representative rows from **results/v3/demos/smartgrid_v3_demo_episode_baseline_v3_2026-05-18.csv**.

Table 13: Representative rows from the baseline V3 demo episode. Action names are added for readability.

Step	Batt.	Demand	Ren.	Price	Action name	Grid	Reward	Risk	Level
0	2	1	0	1	Use battery	0	0.50	50.00	medium
2	0	1	0	1	Buy from grid	1	-1.10	50.00	medium
7	0	2	1	1	Buy from grid	1	0.28	70.81	high
9	0	2	3	0	Sell surplus	0	3.30	50.00	medium
14	1	2	3	0	Store surplus	0	3.12	29.19	low
15	2	1	3	0	Store surplus	0	1.53	15.56	low
18	0	3	1	2	Buy from grid	2	-0.92	70.81	high
19	0	3	0	2	Buy from grid	3	-3.32	70.81	high
87	4	2	3	0	Sell surplus	0	4.33	15.56	low

The selected rows illustrate several behaviors. In step 0, action 0 means *Use battery*; the battery is available and demand is low, so the action covers demand without buying from the grid. In steps 2

and 7, action 1 means *Buy from grid*; this appears when demand cannot be fully covered by local renewable generation and battery storage. In step 9, action 3 means *Sell surplus*, which is possible because renewable generation exceeds demand. In steps 14 and 15, action 2 means *Store surplus*; the battery increases when surplus and capacity are available. Finally, step 87 shows action 3 with high battery and renewable surplus, producing a strong positive reward in a low-risk context.

Negative rewards appear when grid purchases, high risk or invalid actions dominate the step outcome. Positive rewards appear when demand is covered with useful surplus handling and lower risk.

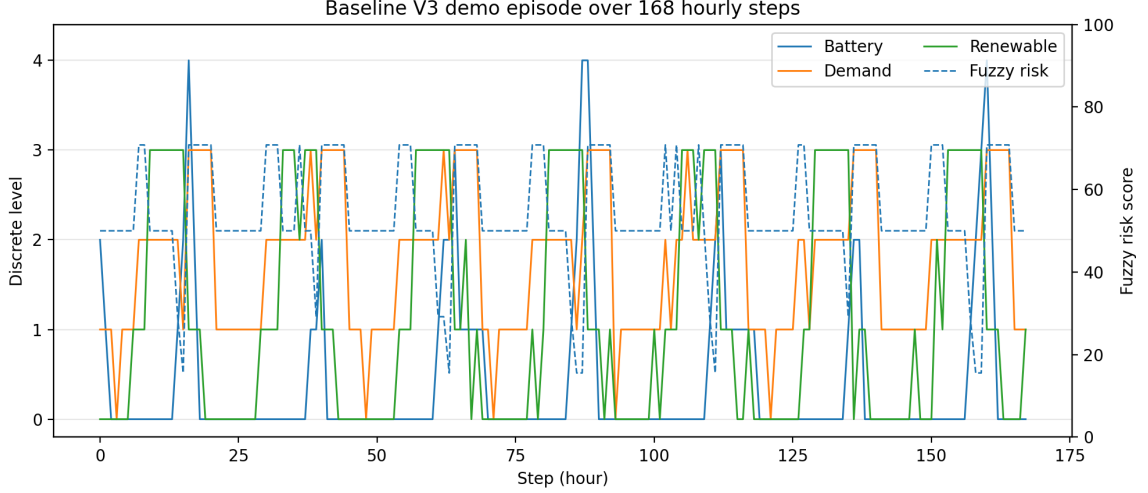


Figure 5: Temporal evolution of the baseline V3 greedy demo episode. The figure illustrates how demand, renewable availability, battery state and fuzzy risk vary across the 168-hour synthetic week. The left axis uses discrete state levels, while the right axis uses the fuzzy risk scale from 0 to 100.

13 Dashboard

The V3 dashboard is implemented with Streamlit in `src/dashboard/app.py`. It reads the latest V3 evaluation summary from `results/v3/summaries/` and available demo episodes from `results/v3/demos/`. The dashboard displays the summary table, reward charts, selected metrics and detailed demo episode trajectories.

The dashboard maps action IDs to human-readable action names: 0 is *Use battery*, 1 is *Buy from grid*, 2 is *Store surplus*, 3 is *Sell surplus* and 4 is *Hold*. This improves interpretability because the reviewer can inspect the behavior of a trained policy without mentally translating numeric action IDs. In the final dashboard, coverage and fuzzy risk are visualized separately: coverage is expressed as a percentage, while fuzzy risk is shown as a 0–100 score rather than as the same physical unit.

14 V3.1 PSO reward-weight tuning

V3.1 is an optional extension that connects Particle Swarm Optimization (PSO) to the V3 reward-design problem [4]. It is not the main learning algorithm. Q-learning remains the method that learns the policy. PSO does not control the environment directly. Instead, each particle represents a candidate vector of reward weights.

The vector order is:

`[demand_covered, unmet_demand, grid_bought, sold, invalid_action, wasted_renewable, risk]`

For each candidate vector, the PSO script creates a temporary V3 environment, trains a temporary Q-learning agent, evaluates the greedy policy and computes a fitness score. Because `pyswarm` minimizes its objective, the implementation minimizes negative fitness.

The V3.1 experiment is included to demonstrate how an evolutionary algorithm can be connected to the V3 reward-design problem. It should be interpreted as a representative reward-weight tuning experiment rather than as a definitive optimization study.

14.1 PSO configuration and bounds

The implementation uses the following bounds for the PSO search space:

Table 14: Reward-weight search bounds used by V3.1 PSO.

Weight	Lower bound	Upper bound
demand_covered	1.000	4.000
unmet_demand	-8.000	-2.000
grid_bought	-2.000	-0.200
sold	0.100	2.000
invalid_action	-4.000	-0.500
wasted_renewable	-2.000	-0.100
risk	-0.080	-0.005

The final V3.1 artifact corresponds to a moderate PSO reward-weight tuning run on `baseline_v3`. It is more representative than the initial smoke test because it evaluates a larger swarm and more PSO iterations, with 250 temporary Q-learning episodes per candidate. Even so, it remains an academic tuning experiment rather than an exhaustive optimization campaign. The recorded hyperparameters are shown in Table 15.

Table 15: Recorded V3.1 PSO hyperparameters.

Parameter	Value
Scenario	<code>baseline_v3</code>
Swarmsize	5
Omega	0.5
<code>phip</code>	1.5
<code>phig</code>	1.5
Max iterations	5
Temporary Q-learning episodes per candidate	250
Best fitness	1027.7971
Objective minimum	-1027.7971

14.2 Best reward weights and metrics

Table 16: Best reward weights recorded in the V3.1 PSO artifact.

Weight	Value
demand_covered	4.0000
unmet_demand	-2.0000
grid_bought	-0.2000
sold	2.0000
invalid_action	-0.5000
wasted_renewable	-0.1000
risk	-0.0050

Table 17: Metrics reported for the recorded V3.1 PSO candidate.

Metric	Value
Average reward	1053.5564
Coverage	93.25%
Average unmet demand	21.00
Average grid bought	146.00
Average sold	36.00
Average wasted renewable	6.00
Average battery end	0.00
Average risk score	57.43
Invalid action rate	16.67%

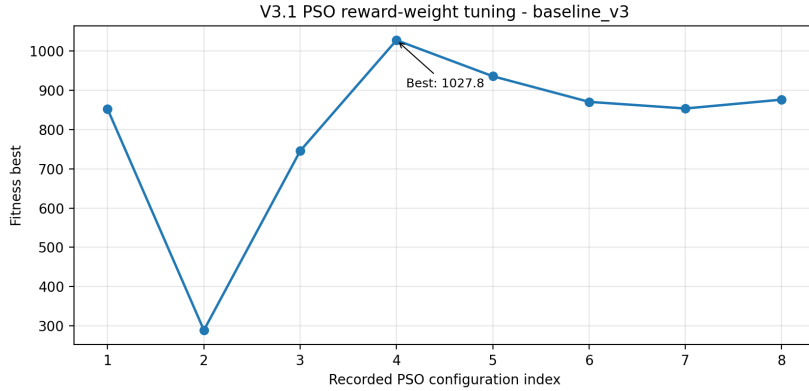


Figure 6: Recorded V3.1 PSO fitness trace for **baseline_v3** using the final moderate PSO configuration. The experiment demonstrates reward-weight tuning through Particle Swarm Optimization, but it should not be interpreted as a definitive optimization campaign or as a replacement for Q-learning.

The final V3.1 PSO artifact uses a moderate configuration with 250 temporary Q-learning episodes per candidate, 5 PSO iterations and a swarm size of 5. This makes the artifact more representative than the initial smoke test while still keeping the experiment computationally manageable. The best candidate improves the recorded PSO fitness compared with the initial smoke run, but the result should not be interpreted as a direct improvement over the main V3 evaluation because it uses a different reward-weight vector. The purpose of V3.1 is to demonstrate how an evolutionary algorithm can be connected to reward-weight tuning, not to replace the main Q-learning pipeline.

15 Course concepts covered

Table 18 summarizes how the final implementation connects to the main Artificial Intelligence course concepts used in the project.

Table 18: Artificial Intelligence course concepts covered by SmartGrid-ES.

Course concept	Implementation in SmartGrid-ES
Reinforcement Learning	Tabular Q-learning agent trained through agent-environment interaction.
Markov Decision Process	Finite state space, finite action space, transition logic and scalar reward.
Gymnasium	Custom <code>SmartGridEnvV3</code> environment with <code>reset()</code> and <code>step()</code> .
Fuzzy logic	Energy-risk module implemented with <code>scikit-fuzzy</code> , membership functions and fuzzy control rules.
Evolutionary algorithms	Optional PSO reward-weight tuning experiment in V3.1.
Experimental evaluation	Scenario-based metrics, plots, demo episodes and Streamlit dashboard.

16 Limitations

The main limitations of the final project are:

Several design notes clarify the scope of these limitations. V3 uses deterministic synthetic CSV time series, and each episode follows a fixed 168-hour weekly sequence. As a result, one greedy evaluation episode is sufficient to reproduce the scenario metrics, because repeated evaluation episodes follow the same sequence from the same initial state. All V3 scenarios also start with the same initial battery level. This is intentional and makes the comparison fair: differences in results are driven by demand, renewable, price and weather profiles rather than by different initial conditions. Finally, `weather` is included in the observation vector as contextual scenario information. In V3, it does not directly modify the transition equations or the reward; a future V4 could introduce direct causal weather effects.

- The input data are synthetic and controlled.
- The environment is discrete and simplified.
- The project does not simulate a real electricity market or market settlement mechanism.
- The environment does not model real grid physics, power flows, voltage constraints or network topology.
- There is no live data ingestion or external real-time API.
- The learning algorithm is tabular Q-learning, not deep reinforcement learning.
- The project does not implement real multi-agent reinforcement learning.
- The PSO extension is an optional moderate reward-weight tuning experiment, not the main control algorithm and not an exhaustive optimization campaign.
- The tabular approach is interpretable but limited to small discrete state spaces.

These limitations are deliberate design choices within the academic scope of the project, intended to keep the system interpretable, reproducible and suitable for analysis.

17 Conclusions

V3 is the strongest and most coherent version of SmartGrid-ES. It moves beyond the wrapper-based design of V2 and introduces a custom Gymnasium environment tailored to the simplified smart-grid problem. This makes the state variables, action semantics, transition logic, reward function, scenario data and evaluation outputs explicit.

The tabular Q-learning agent learns policies that can be evaluated across multiple synthetic scenarios. The evaluation results show that scenarios with different demand, renewable and price patterns produce different reward outcomes even when coverage remains high. The results also show that demand coverage alone is not sufficient to evaluate the learned policy. Scenarios with high coverage can still obtain negative rewards when grid purchases, fuzzy risk, wasted renewable energy or invalid actions accumulate.

The fuzzy risk module improves interpretability by converting battery, demand, renewable and price conditions into a readable risk signal. The dashboard further supports inspection by displaying summary metrics, episode evolution and human-readable action names.

The optional V3.1 PSO experiment demonstrates how an evolutionary algorithm can be connected to reward-weight tuning. It does not replace Q-learning, does not directly control the environment and should not be presented as the main control method. Overall, the project aligns with the Artificial Intelligence course topics: Gymnasium environments, reinforcement learning, Markov decision processes, fuzzy logic, evolutionary algorithms and experimental evaluation.

18 Future work

Possible extensions include:

- using real or semi-real demand, weather and price datasets,
- evaluating longer temporal windows beyond one synthetic week,
- adding stochastic scenario generation,
- introducing direct causal weather effects on transitions or rewards,
- enriching the battery model with charge/discharge efficiency and degradation,
- moving from discrete to continuous observations or actions,
- implementing DQN or other deep RL baselines,
- exploring a true multi-agent extension with independent producers, consumers and storage agents,
- running a systematic PSO or genetic-algorithm tuning campaign with larger search budgets.

A Execution commands

```
python -m scripts.generate_synthetic_timeseries
python -m src.training.train --version v3 --all-v3
python -m src.training.evaluate --version v3 --all-v3
streamlit run src/dashboard/app.py
```

For the optional V3.1 PSO experiment, the final report uses the moderate artifact generated with:

```
python -m src.optimization.pso_reward_tuning --scenario baseline_v3 \
--episodes 250 --maxiter 5 --swarmsizes 5 \
--omegas 0.5,0.7 --phips 1.0,1.5 --phigs 1.0,1.5
```

The initial smoke command remains useful only as a quick integration check and is not the final PSO result reported in Section 14.

B Pinned dependencies

The final repository pins the main runtime dependencies to support reproducibility:

Table 19: Pinned dependencies in the final `requirements.txt`.

Package	Version
gymnasium	1.3.0
numpy	2.4.1
matplotlib	3.10.9
pandas	3.0.1
scikit-fuzzy	0.5.0
streamlit	1.57.0
pyswarm	0.8.0

C Repository structure relevant to V3

Table 20: Repository areas most relevant to the final V3 delivery.

Area	Path	Purpose
Data	<code>data/scenarios/</code>	JSON scenario definitions for V3.
Data	<code>data/timeseries/</code>	Synthetic training and evaluation CSV inputs.
Main results	<code>results/smartgrid_es_v3_report.pdf</code>	Final academic report PDF.
Main results	<code>results/v3/models/</code>	Final trained Q-table models.
Main results	<code>results/v3/demos/</code>	Greedy 168-step demo episodes.
Main results	<code>results/v3/summaries/</code>	Final evaluation summary CSV.
Main results	<code>results/v3/plots/</code>	V3 result plots.
Optional PSO	<code>results/v3_1/best_configs/</code>	Best reward-weight JSON from the final moderate PSO run.
Optional PSO	<code>results/v3_1/summaries/</code>	PSO reward-tuning summary CSV.
Optional PSO	<code>results/v3_1/plots/</code>	PSO result plot.
Source code	<code>src/envs/</code>	Environments, loaders, reward and fuzzy risk logic.
Source code	<code>src/agents/</code>	Tabular Q-learning agent.
Source code	<code>src/training/</code>	Training and evaluation entry points.
Source code	<code>src/optimization/</code>	Optional PSO reward-weight tuning.
Source code	<code>src/dashboard/</code>	Streamlit dashboard.

D Final artifacts and reproducibility note

Table 21: Final artifacts intended for reviewer inspection.

Artifact	Path	Purpose
Final report PDF	<code>results/smartgrid_es_v3_report.pdf</code>	Academic report for delivery.
Q-tables	<code>results/v3/models/</code>	Trained tabular policies.
Demo episodes	<code>results/v3/demos/</code>	Greedy weekly trajectories.
Evaluation summary	<code>results/v3/summaries/</code>	Aggregate metrics by scenario.
V3 plots	<code>results/v3/plots/</code>	Visual comparison of main results.
PSO outputs	<code>results/v3_1/</code>	Optional reward-weight tuning artifacts.

Reproducibility note. Final V3 artifacts are stored under `results/v3/`, while optional PSO artifacts are stored under `results/v3_1/`. The final report PDF is intended to be placed under

`results/smartgrid_es_v3_report.pdf`. The main reproduction commands are listed above. The configured seed is 42, and each V3 episode contains 168 hourly steps following a deterministic synthetic weekly sequence. The PSO artifacts in `results/v3_1/` correspond to the optional reward-weight tuning layer, not to the main V3 fixed-weight evaluation.

E Main files and artifacts used

The report was prepared from the following repository files and artifacts:

Source files	Data and result artifacts
<code>README.md</code>	<code>data/scenarios/*.json</code>
<code>src/envs/smartgrid_env_v3.py</code>	<code>data/timeseries/*.csv</code>
<code>src/envs/reward_v3.py</code>	<code>results/v3/summaries/smartgrid_v3_evaluation_summary_2026-05-18.csv</code>
<code>src/envs/fuzzy_risk.py</code>	<code>results/v3/demos/smartgrid_v3_demo_episode_baseline_v3_2026-05-18.csv</code>
<code>src/envs/scenario_loader.py</code>	<code>results/v3/plots/*.png</code>
<code>src/envs/timeseries_loader.py</code>	<code>results/v3_1/summaries/pso_reward_tuning_baseline_v3_2026-05-18.csv</code>
<code>src/envs/smartgrid_env.py</code>	<code>results/v3_1/best_configs/best_reward_weights_baseline_v3_2026-05-18.json</code>
<code>src/envs/factory.py</code>	<code>results/v3_1/plots/pso_reward_tuning_baseline_v3_2026-05-18.png</code>
<code>src/agents/qlearning_agent.py</code>	
<code>src/training/config.py</code>	
<code>src/training/train.py</code>	
<code>src/training/evaluate.py</code>	
<code>src/training/compare_v2_v3.py</code>	
<code>src/optimization/pso_reward_tuning.py</code>	
<code>src/dashboard/app.py</code>	

References

- [1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018.
- [2] C. J. C. H. Watkins and P. Dayan, “Q-learning,” *Machine Learning*, vol. 8, pp. 279–292, 1992.
- [3] Farama Foundation, *Gymnasium Documentation*. Available: <https://gymnasium.farama.org/>.
- [4] J. Kennedy and R. Eberhart, “Particle swarm optimization,” in *Proceedings of ICNN’95 - International Conference on Neural Networks*, 1995, pp. 1942–1948.
- [5] J. D. Warner et al., *scikit-fuzzy Documentation*. Available: <https://pythonhosted.org/scikit-fuzzy/>.
- [6] L. A. Zadeh, “Fuzzy sets,” *Information and Control*, vol. 8, no. 3, pp. 338–353, 1965.